# -Design Doc: AI Competitive Intelligence Platform

**Status:** Draft **Author:** [Your name] **Last updated:** August 2026 **Reviewers:** —

---

## 1. Summary

A user pastes a product URL. The system autonomously researches that product, discovers its competitors, gathers public evidence about each, and produces an evidence-backed competitive analysis where every material claim links to a retrievable source.

The core bet is that the bottleneck in competitive analysis is not writing the report — LLMs are already good at that — but **grounding it**. Analysts spend their time finding sources, and readers spend their time deciding whether to trust conclusions. This system optimizes for source traceability over prose quality, and treats "confidently wrong" as the primary failure mode rather than "incomplete."

---

## 2. Problem

PMs, founders, and strategy teams redo the same manual research constantly:

- Find competitors (search, ask around, check review-site categories, read G2 grids)
- Open 20-40 tabs across marketing sites, pricing pages, changelogs, review sites, news
- Manually normalize inconsistent feature and pricing language into a comparison
- Write it up, then watch it go stale in six weeks

This takes 1–2 days for a decent first pass. It's repeated per product, per quarter, per team, with no shared artifact and no provenance, six months later nobody can tell you where the "$49/seat" number came from or whether it was ever true.

Generic LLM chat is a poor substitute. Ask a model for competitors and it produces plausible names from training data with no freshness guarantee, invented pricing, and no citations. The output is unusable for a decision anyone has to defend.

### Why now

Three things make this tractable that weren't 18 months ago: tool-calling reliability is high enough for multi-step research loops; MCP gives a standard interface for exposing research

tools; and vector search over a curated corpus is cheap enough to run per-report rather than as a batch pipeline.

---

## 3. Goals and non-goals

### Goals

| # | Goal | Success measure |
|---|---|---|
| G1 | Discover relevant competitors from a URL alone | ≥70% precision, ≥60% recall vs. human-labeled set |
| G2 | Every material claim traceable to a source | ≥90% of claims carry a working, on-point citation |
| G3 | Clearly separate evidence from interpretation | Distinct visual treatment; 100% of interpretive claims labeled |
| G4 | Beat manual research on time | <10 min to full report vs. 1–2 days |
| G5 | Measurable quality | Eval suite over ≥20 benchmark products, run in CI |

---

## 4. Users and use cases

**Primary: the PM doing a competitive readout.** Needs a defensible artifact to bring to a roadmap review. Cares most about citations, because they'll be challenged on the numbers.

**Secondary: founder validating a market.** Needs the landscape shape and pricing bands. Cares most about discovery — the competitors they *didn't* know about.

**Tertiary: candidate prepping for interviews.** Needs to sound informed about a company's competitive position quickly.

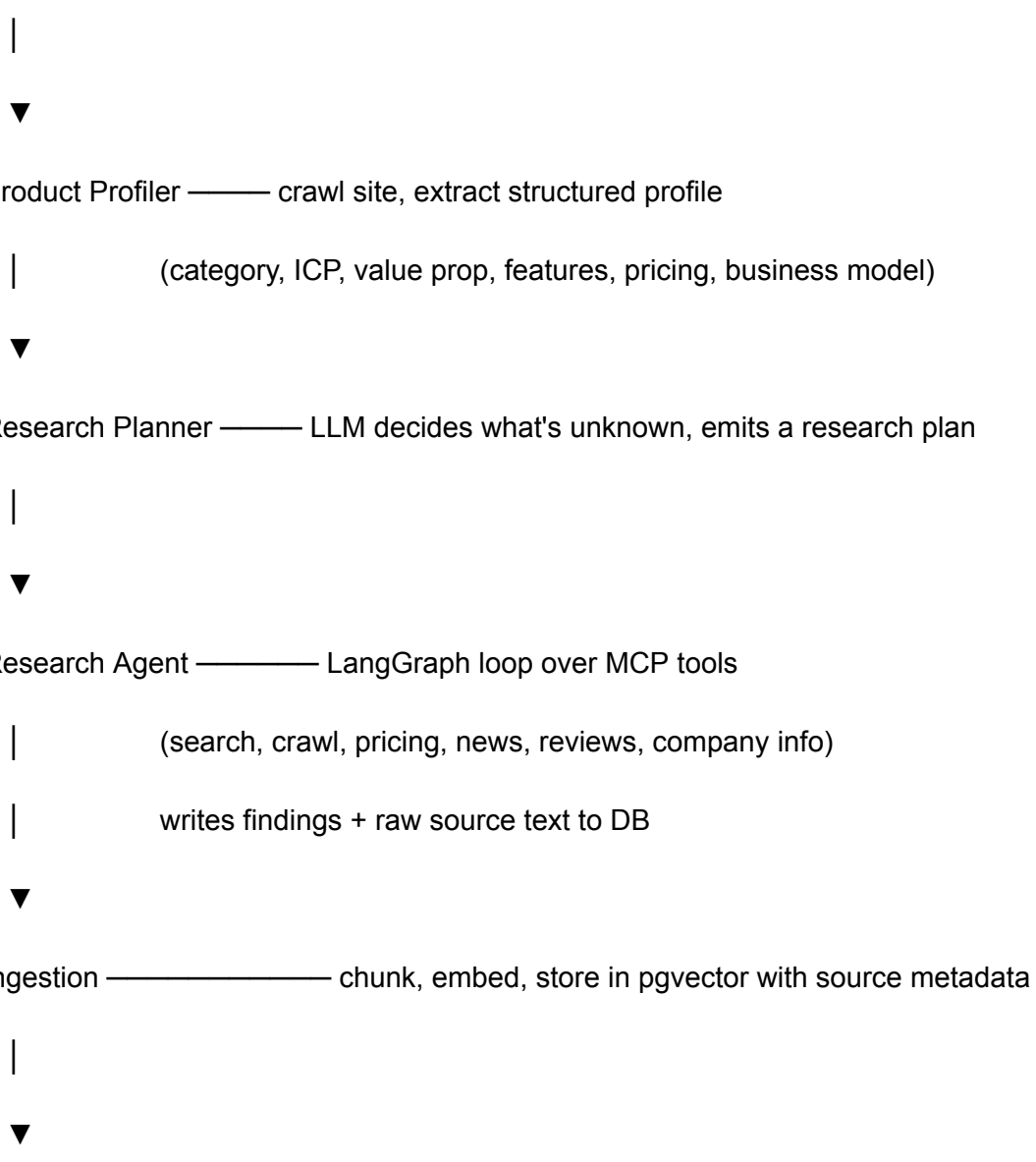
**Key insight from these three:** all of them are going to *check the work*. Nobody uses this output raw. So the product's job is to make checking fast, not to make checking unnecessary. This is why source attribution is a P0 and prose polish is a P2.

---

# 5. Proposed solution

## 5.1 High-level flow

User enters product URL



[5] Analysis Agent ———— per report section: retrieve → synthesize → cite

[6] Dashboard ———————— sections rendered progressively as they complete

## 5.2 Stage detail

**[1] Product Profiler.** Fetch the URL plus obvious subpages (`/pricing`, `/features`, `/about`, `/customers`). Extract a structured `ProductProfile` via a constrained JSON schema. This profile is the seed for everything downstream, so it gets its own eval — a wrong category poisons the whole run.

**[2] Research Planner.** Given the profile, the model emits an explicit plan: which competitor-discovery strategies to run, which dimensions to research, what "done" looks like. Making the plan a first-class visible object (not an implicit chain) is a deliberate choice — it's inspectable, it's loggable, and it lets the UI show the user what the agent is doing while they wait.

**[3] Research Agent.** A LangGraph state machine, not a free-running ReAct loop. Nodes: `discover_competitors` → `dedupe_and_rank` → `research_competitor` (fan-out) → `check_coverage` → `finalize`. Bounded by max iterations, max tool calls, and a cost ceiling. Every tool result is persisted with its URL, fetch timestamp, and raw text before any model touches it.

**[4] Ingestion.** Chunk (~800 tokens, 100 overlap), embed, store. Every chunk carries `source_url`, `fetched_at`, `source_type`, `competitor_id`. Source type matters for trust weighting later.

**[5] Analysis Agent.** One retrieval-and-synthesis pass per report section, each with a section-specific query strategy. Output is structured JSON with claim-level citations, not free prose — the frontend renders it. Uncited claims are permitted only in explicitly-labeled interpretation blocks.

## 5.3 Competitor discovery — the hard part

This deserves its own section because it's where the product lives or dies, and where a naive implementation fails most visibly.

Multi-strategy, then merge:

1. **LLM prior** — ask the model directly. Fast, decent recall for established categories, useless for anything younger than the training cutoff. Treated as a *candidate generator only*, never as evidence.
2. **Search-based** — query "alternatives to X", "X vs", "best [category] tools". Listicles and comparison pages are noisy but high-recall.
3. **Backlink/mention mining** — who does the product's own site compare itself to? Comparison pages are a strong signal because they're self-declared.
4. **Category traversal** — from the product's category, enumerate other products in it.

Merge with fuzzy name matching plus domain normalization, then **verify**: every candidate must have a reachable homepage confirming it operates in the stated category. Unverified candidates are dropped, not surfaced with a caveat.

Then classify direct vs. indirect vs. adjacent, and rank by relevance. Cap at 5.

**Open question (O1):** "Competitor" is genuinely fuzzy — is Notion a competitor to Jira? Ground truth is contested, which makes recall hard to measure honestly. Current plan is to have labelers mark candidates on a 3-point scale (clearly yes / defensible / clearly no) and score against "clearly yes" for recall and "clearly no" for precision, treating the middle band as neutral. This is a compromise and it's worth saying so in the writeup.

## 5.4 MCP server design

A custom MCP server exposes research capabilities as discoverable tools. The point isn't that MCP is required here — a plain function registry would work — it's that MCP makes the tool surface portable and independently testable. Say that out loud rather than pretending it's load-bearing.

**Tools:**

| Tool | Input | Output |
|---|---|---|
| `search_web` | query, recency filter | ranked results with snippets |
| `crawl_page` | url, extraction hint | cleaned text + metadata |
| `extract_pricing` | url | structured tiers, prices, billing period, feature gates |
| `get_company_info` | name or domain | funding, headcount, founded, HQ |
| `search_news` | entity, date range | articles with dates and outlets |

| Tool | Input | Output |
|---|---|---|
| `get_reviews_summary` | product name | aggregate sentiment + themes (compliant sources only) |
| `find_alternatives` | product name/url | candidate competitor list |

**Resources:** `products/{id}/profile`, `research/{run_id}/findings`, `benchmarks/labeled_set`

Design principles: every tool returns source URL and fetch timestamp; every tool has a token budget and truncates deterministically; tools fail soft with structured errors so a single dead site doesn't kill a run.

## 5.5 Data model

**products** (id, url, name, category, profile_json, created_at)

**runs** (id, product_id, status, started_at, finished_at, cost_cents, token_count, plan_json)

**competitors** (id, run_id, name, domain, relationship, -- direct|indirect|adjacent confidence, discovery_method, verified)

**sources** (id, run_id, url, source_type, fetched_at,

raw_text, http_status, content_hash)

**chunks** (id, source_id, text, embedding vector(1536), metadata_json)

**findings** (id, run_id, competitor_id, dimension, value_json,

source_ids[], extracted_at)

**claims** (id, run_id, section, text, claim_type, -- evidence|interpretation

source_ids[], confidence)

**eval_results** (id, run_id, metric, score, details_json)

`claims` as a separate table (rather than embedding citations in rendered text) is what makes G2 and G3 measurable. You can query "what fraction of claims in this run have ≥1 source" directly.

### 5.6 Attribution and trust

Three tiers, rendered distinctly:

- **Verified** — extracted from a primary source (the competitor's own pricing page). Highest trust.
- **Reported** — from a secondary source (news, review site, listicle). Medium.
- **Interpretation** — model synthesis across evidence. Explicitly labeled, never presented as fact.

Recommendations are always interpretation, and every recommendation must name the evidence claims it rests on. A recommendation with no supporting claims is a bug, and the eval suite checks for it.

**Staleness** is surfaced per claim: pricing fetched 40 days ago is shown with its date. Cheap to implement, disproportionately valuable for trust.

---

## 6. Architecture

**Frontend** — Next.js / React / TypeScript on Vercel. Server-sent events for progressive rendering.

**Backend** — Python / FastAPI. Async job queue for runs (a run is minutes long, so the API returns a run ID immediately and streams status).

**Orchestration** — LangGraph. Chosen over a hand-rolled loop for checkpointing and resumability: a 6-minute run that dies at minute 5 must not restart from zero.

**Storage** — Supabase Postgres + pgvector. One database for app data and vectors keeps the stack small.

**Models** — a larger model for planning and final synthesis, a cheaper/faster one for extraction and classification. Extraction is high-volume and schema-constrained, so it's the obvious place to save money.

**Observability** — LangSmith for traces. Non-optional: debugging a 40-tool-call agent run from logs alone is miserable.

# 7. Evaluation

The eval framework is arguably the most differentiating part of this project. Anyone can build an agent that produces a report; showing it's *good* is rarer.

## 7.1 Benchmark set

20–25 products chosen for coverage: well-known SaaS (easy), niche B2B (hard discovery), recently launched (post-cutoff, tests whether research beats priors), ambiguous-category products (tests classification), and 2–3 non-English or non-US products (tests failure modes honestly).

For each, hand-label: expected competitors on the 3-point scale from §5.3, correct category, and ground-truth pricing for the top 3 competitors as of a fixed date.

## 7.2 Metrics

| **Metric** | **Definition** | **Target** |
|---|---|---|
| Competitor precision | fraction surfaced that aren't "clearly no" | ≥0.70 |
| Competitor recall | fraction of "clearly yes" surfaced | ≥0.60 |
| Category accuracy | exact or acceptable-synonym match | ≥0.85 |
| Pricing extraction accuracy | tier + price both correct | ≥0.80 |
| Citation validity | citation resolves and supports the claim | ≥0.90 |
| Hallucination rate | claims contradicted by their own cited source | ≤0.05 |
| Recommendation groundedness | recs tracing to ≥1 evidence claim | 1.00 |
| Cost per run | mean | ≤$1.20 |
| P50 / P95 latency | end to end | ≤7 min / ≤12 min |

### 7.3 How it's judged

Citation validity and hallucination rate use **LLM-as-judge with a spot-checked human sample** — judge the claim against the retrieved source text, then manually verify ~15% of judgments to estimate judge error. Report the judge's own agreement rate alongside the metric; a judged metric without that number is not credible.

Deterministic metrics (precision, recall, cost, latency) run in CI on every merge to main. Judged metrics run on demand, since they cost money.

### 7.4 What to publish

Keep a results table across versions in the repo. "V1 → V3 improved citation validity 0.71 → 0.93 by adding source verification before synthesis" is the single most valuable sentence this project can generate. Regressions should be shown too — a table with only improvements looks curated.

---

## 8. Risks and mitigations

| Risk | Severity | Mitigation |
|---|---|---|
| **Scope.** As specified, this is 4–6 months of full-time work | High | Ruthless V1 cut (§9). Ship narrow, iterate publicly |
| **Crawling legality.** Many review sites prohibit scraping in ToS | High | Use official APIs and search-engine APIs; honor `robots.txt`; no login-gated content; document what's excluded and why |
| **Fuzzy ground truth** for competitor sets | Medium | 3-point labeling, neutral middle band, report inter-labeler agreement |
| **Hallucinated pricing.** Pricing pages are dynamic, JS-heavy, region-varying | Medium | Extract only from primary sources; store raw HTML; mark unparseable as "unavailable" rather than guessing |

| Risk | Severity | Mitigation |
|---|---|---|
| **Cost runaway** on a looping agent | Medium | Hard per-run ceiling, iteration cap, cached crawls |
| **Agent thrash** — loops re-searching the same thing | Medium | Explicit state machine over free ReAct; coverage check node; visited-URL set |
| **Freshness illusion.** Users assume the report is current | Low | Per-claim timestamps, prominent run date |

---

## 9. Milestones

The full spec is too large for a portfolio project. This is the cut I'd defend in an interview — and being able to explain the cut is itself the signal.

**V1 — Grounded single-shot report (2–3 weeks of evenings). This is the resume artifact.**

- Profiler, discovery (strategies 1–3), 5 competitors max
- Four sections: exec summary, landscape, feature matrix, pricing
- Full citation model with the three trust tiers
- MCP server with 4 tools: search, crawl, pricing, alternatives
- Eval on 10 products, deterministic metrics only
- Deploy, demo video, writeup

**V2 — Depth (1–2 weeks)**

- Remaining sections: sentiment, positioning, trends, threats, recommendations
- Full 7-tool MCP server, discovery strategy 4
- Full 20-product benchmark, judged metrics, LangSmith traces
- Cost and latency dashboards

**V3 — Monitoring (optional, 2 weeks)**

- Scheduled re-crawls using `content_hash` diffing
- Change classification and strategic-impact explanation
- Notifications

If time runs short, **cut V2 sections before cutting V1 evals.** A four-section report with a rigorous eval suite is a stronger artifact than a nine-section report with none. The eval suite is the thing other candidates won't have.

---

## 10. Open questions

- **O1.** How to handle contested competitor ground truth — is the neutral-middle-band approach defensible, or does it inflate precision? (§5.3)
- **O2.** Should indirect competitors get the same research depth as direct ones, or a lighter pass? Depth is expensive and indirect competitors are lower-value.
- **O3.** Is LLM-as-judge credible enough for citation validity, or does the human sample need to be larger than 15%?
- **O4.** What's the right behavior when discovery finds fewer than 3 competitors — is that a genuine finding about a new category, or a failure? They look identical to the system.
- **O5.** Should users be able to add or remove competitors manually? Improves output quality; muddies the eval story, since you can no longer attribute report quality purely to the system.

---

## Appendix A: Portfolio artifacts

The code is the ticket; these are what get discussed in the interview.

1. **This design doc**, kept current — including the parts that turned out wrong
2. **Eval results table** across versions, regressions included
3. **A tradeoff writeup**: why LangGraph over raw ReAct, why MCP when a function registry would do, why cap at 8 competitors, why a claims table instead of inline citations
4. **A failure gallery** — 3–5 real runs that went badly, with root cause. Counterintuitively the strongest artifact here; it's the clearest evidence of actual product judgment
5. **Cost and latency numbers**, measured not estimated
6. **A 3-minute demo video** — assume nobody runs the code